Name: Kamal Sharma

UID: 24BAI70380

Course: BE-CSE (AI&ML)

Subject: Database Management System

Experiment: Implementation of Database Triggers

1. Aim of the Session

To understand and implement database triggers in PostgreSQL to automate data validation and computational logic, ensuring data integrity by enforcing business rules during DML operations.

2. Software Requirements

- **Database**:
    - Oracle live SQL
    - PostgreSQL Database (PgAdmin)

3. Objective of the Session

By the end of the session, the following objectives were achieved:

- Create a trigger function that performs automatic calculations on row-level data.
- Define a **BEFORE INSERT** trigger to intercept data entry for validation.
- Implement custom exception handling using the RAISE EXCEPTION command.
- Verify trigger behaviour by testing both valid and invalid data scenarios within a transaction.

4. Practical / Experiment Steps

The experiment was carried out through the following activities:

- **Schema Setup:** Created the employee table with fields for working hours, hourly rate, and a computed payable amount.
- **Trigger Function Development:** Designed the PL/pgSQL function CALCULATE_AMOUNT() to compute payable salary dynamically.
- **Validation Logic:** Added a conditional check to restrict total payable amount to ≤ 25,000.
- **Trigger Binding:** Configured a row-level trigger CAL_PAYABLE_AMOUNT to execute before every insert.
- **Testing:** Used anonymous DO blocks to insert valid and invalid records, verifying exception handling and trigger enforcement.

5. Procedure of the Practical

- Execution was performed in the following order:
- Accessed PostgreSQL console and created the employee table.
- Defined the trigger function using NEW keyword to compute total_payable_amount.
- Implemented validation logic with RAISE EXCEPTION for amounts > 25,000.
- Created the trigger CAL_PAYABLE_AMOUNT with BEFORE INSERT and FOR EACH ROW.

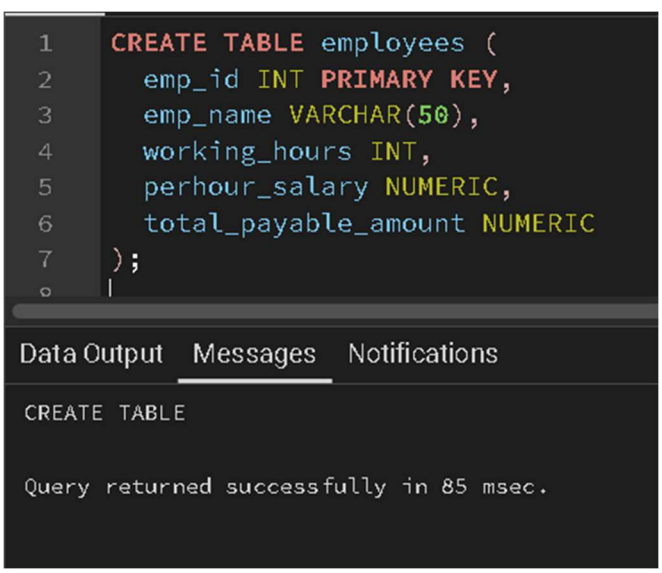
- Inserted a valid record (amount < 25,000) and verified computed results.
- Attempted an invalid record (amount > 25,000) to confirm exception handling.
- Checked final table state to ensure invalid data was blocked.

6. I/O Analysis (Input / Output Analysis)

SQL Input Queries

```
CREATE TABLE employees (

  emp_id INT PRIMARY KEY,

  emp_name VARCHAR(50),

  working_hours INT,

  perhour_salary NUMERIC,

  total_payable_amount NUMERIC

);
```

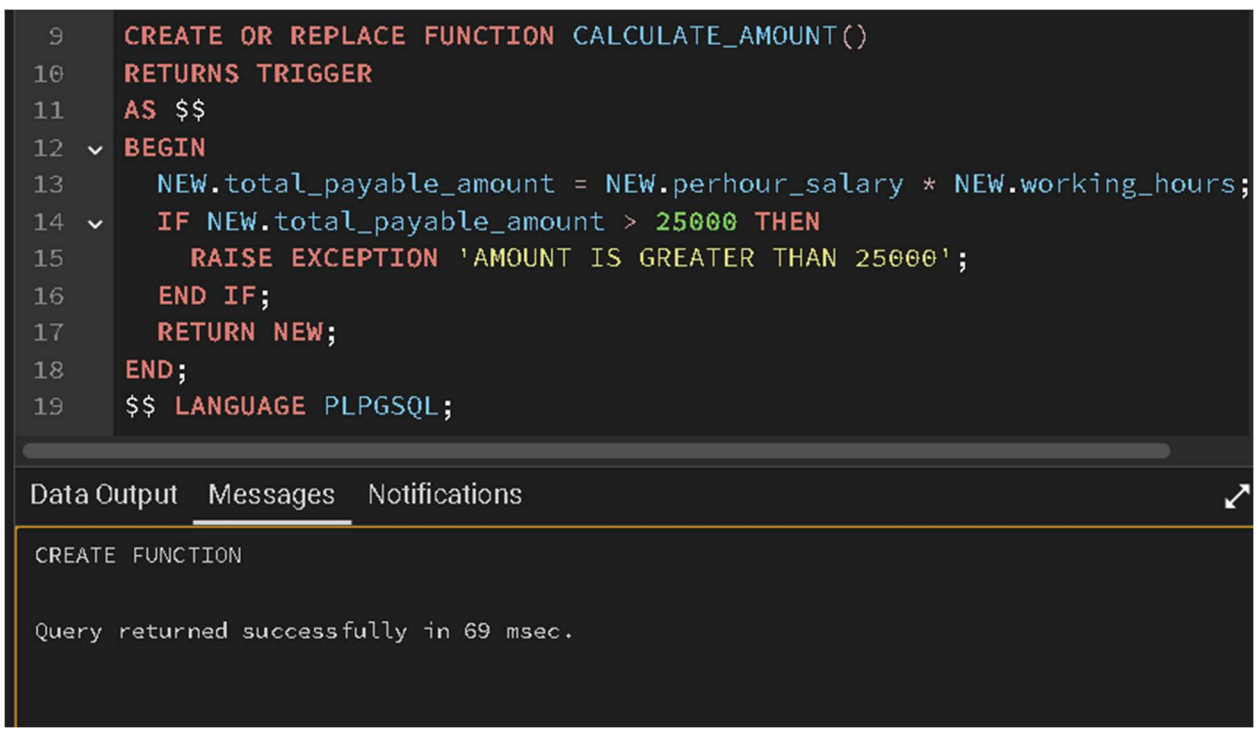
Output :



SQL Input Queries

```
CREATE OR REPLACE FUNCTION CALCULATE_AMOUNT()

RETURNS TRIGGER

AS $$

BEGIN

  NEW.total_payable_amount = NEW.perhour_salary * NEW.working_hours;

  IF NEW.total_payable_amount > 25000 THEN

    RAISE EXCEPTION 'AMOUNT IS GREATER THAN 25000';

  END IF;

  RETURN NEW;

END;

$$ LANGUAGE PLPGSQL;
```

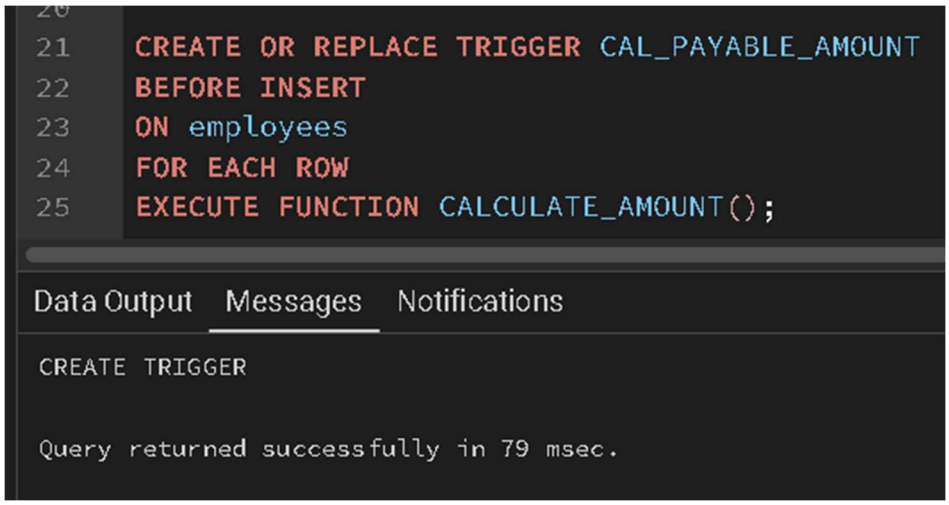
Output :



SQL input Queries

```
CREATE OR REPLACE TRIGGER CAL_PAYABLE_AMOUNT

BEFORE INSERT

ON employees

FOR EACH ROW

EXECUTE FUNCTION CALCULATE_AMOUNT();
```

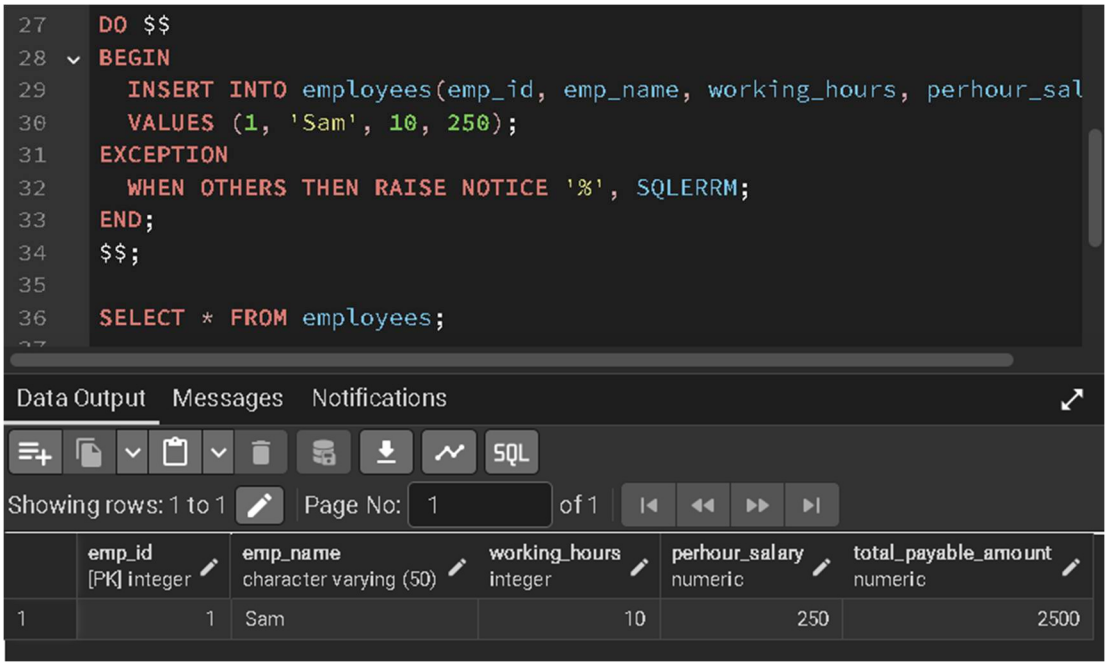
Output :



SQL input Queries

```
DO $$

BEGIN

  INSERT INTO employees(emp_id, emp_name, working_hours, perhour_salary)

  VALUES (1, 'Sam', 10, 250);

EXCEPTION

  WHEN OTHERS THEN RAISE NOTICE '%', SQLERRM;

END;

$$;

SELECT * FROM employees;
```

Output :



SQL input Queries

```
DO $$

BEGIN

  INSERT INTO employees(emp_id, emp_name, working_hours, perhour_salary)

  VALUES (2, 'Amit', 11, 28000);

EXCEPTION

  WHEN OTHERS THEN RAISE NOTICE '%', SQLERRM;

END;

$$;
```

Output :

```
SELECT * FROM employees;

DO $$
BEGIN
  INSERT INTO employees(emp_id, emp_name, working_hours, perhour_sal
  VALUES (2, 'Amit', 11, 28000);
EXCEPTION
  WHEN OTHERS THEN RAISE NOTICE '%', SQLERRM;
END;
$$;
```

Data Output Messages Notifications

```
NOTICE:  AMOUNT IS GREATER THAN 25000
DO

Query returned successfully in 100 msec.
```

7. Learning Outcome

- Understood the difference between **row-level** and **statement-level** triggers.
- Learned how triggers automate calculations and enforce business rules.
- Practiced exception handling with RAISE EXCEPTION for custom validation.
- Verified trigger lifecycle and its role in maintaining data integrity.